MODUL 11 TAHAP 2

TAHAP 2: Implementasi Fitur DBMS (Tipe Data JSON)

Tahap ini mengimplementasikan tipe data JSON pada basis data katalog. Katalog yang dibuat berisi data tanaman dari proyek Green Map PNM, karena atribut tanaman berbeda-beda menurut jenisnya (misalnya pohon memiliki tinggi, sedangkan tanaman hias memiliki warna bunga). Atribut yang beragam seperti ini sesuai disimpan dalam satu kolom JSON tanpa banyak kolom kosong.

DBMS yang digunakan adalah MySQL 8.0.30 yang mendukung tipe data JSON native. Katalog dibuat pada database terpisah, db_katalog_tanaman, sehingga database proyek db_green_map_pnm tidak berubah. Empat operasi yang dikerjakan adalah penyimpanan, pembacaan, pencarian, dan perubahan atribut JSON.

2.1 Membuat Database dan Tabel Katalog

Database db_katalog_tanaman dibuat, kemudian tabel katalog_tanaman dengan kolom detail bertipe JSON untuk menyimpan atribut tanaman. Id tanaman tidak memakai AUTO_INCREMENT karena nilainya disamakan dengan id tanaman pada proyek Green Map.

```
CREATE DATABASE IF NOT EXISTS db_katalog_tanaman;
USE db_katalog_tanaman;

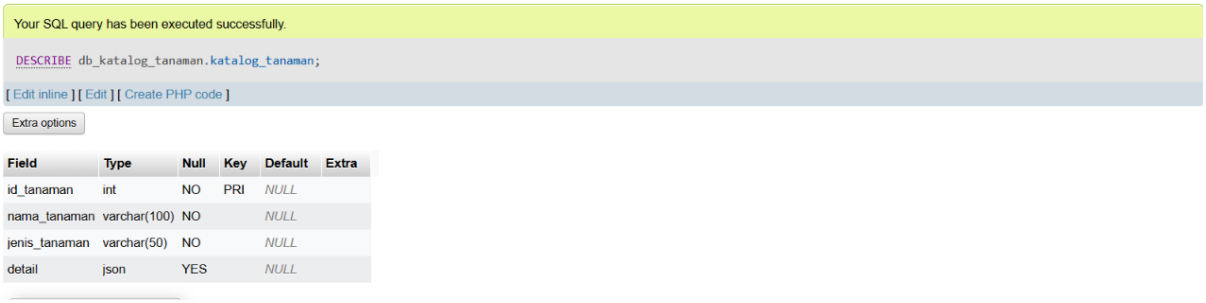
CREATE TABLE katalog_tanaman (
  id_tanaman INT PRIMARY KEY,
  nama_tanaman VARCHAR(100) NOT NULL,
  jenis_tanaman VARCHAR(50) NOT NULL,
  detail JSON
) ENGINE=InnoDB;
```



Gambar 1. Pembuatan database dan tabel katalog_tanaman

Struktur tabel diperiksa dengan perintah DESCRIBE. Hasilnya menunjukkan kolom detail bertipe json, yang membuktikan bahwa tipe data JSON native berhasil diimplementasikan.

```
DESCRIBE db_katalog_tanaman.katalog_tanaman;
```


Your SQL query has been executed successfully.

DESCRIBE db_katalog_tanaman.katalog_tanaman;

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
id_tanaman	int	NO	PRI	NULL	
nama_tanaman	varchar(100)	NO		NULL	
jenis_tanaman	varchar(50)	NO		NULL	
detail	json	YES		NULL	

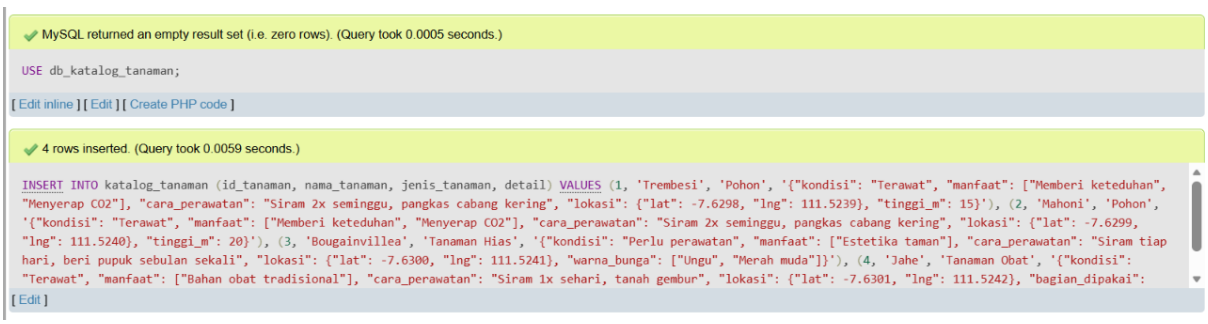
Gambar 2. Struktur tabel katalog_tanaman (kolom detail bertipe json)

2.2 Penyimpanan Data JSON

Empat tanaman dari proyek Green Map dimasukkan ke katalog. Kolom detail berisi kondisi, manfaat (array), cara perawatan, lokasi (objek bersarang berisi lat dan lng), serta atribut khusus per jenis, yaitu tinggi_m untuk pohon, warna_bunga untuk tanaman hias, dan bagian_dipakai untuk tanaman obat.

```
USE db_katalog_tanaman;

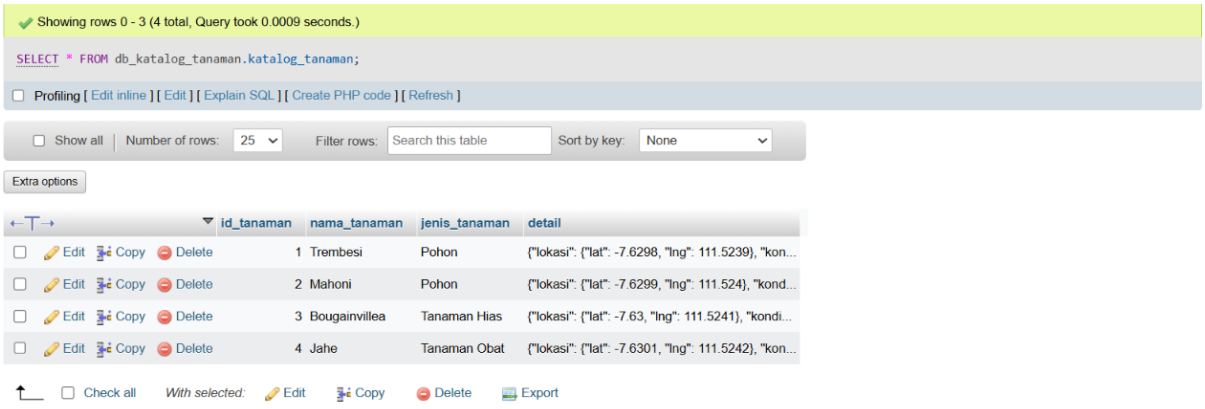
INSERT INTO katalog_tanaman (id_tanaman, nama_tanaman, jenis_tanaman, detail) VALUES
(1, 'Trembesi', 'Pohon',
 '{"kondisi": "Terawat", "manfaat": ["Memberi keteduhan", "Menyerap CO2"], "cara_perawatan": "Siram 2x seminggu, pangkas cabang kering", "lokasi": {"lat": -7.6298, "lng": 111.5239}, "tinggi_m": 15}'),
(2, 'Mahoni', 'Pohon',
 '{"kondisi": "Terawat", "manfaat": ["Memberi keteduhan", "Menyerap CO2"], "cara_perawatan": "Siram 2x seminggu, pangkas cabang kering", "lokasi": {"lat": -7.6299, "lng": 111.5240}, "tinggi_m": 20}'),
(3, 'Bougainvillea', 'Tanaman Hias',
 '{"kondisi": "Perlu perawatan", "manfaat": ["Estetika taman"], "cara_perawatan": "Siram tiap hari, beri pupuk sebulan sekali", "lokasi": {"lat": -7.6300, "lng": 111.5241}, "warna_bunga": ["Ungu", "Merah muda"]}'),
(4, 'Jahe', 'Tanaman Obat',
 '{"kondisi": "Terawat", "manfaat": ["Bahan obat tradisional"], "cara_perawatan": "Siram 1x sehari, tanah gembur", "lokasi": {"lat": -7.6301, "lng": 111.5242}, "bagian_dipakai": "Rimpang"}');
```



Gambar 3. Penyimpanan 4 data tanaman berformat JSON (4 rows inserted)

Data yang tersimpan diperiksa dengan SELECT. Isi kolom detail tampil terpotong karena panjang, dan urutan kunci JSON berbeda dari saat dimasukkan (kunci lokasi tampil lebih awal). Hal ini wajar karena MySQL menyimpan tipe JSON dalam bentuk biner yang sudah dinormalisasi.

```
SELECT * FROM db_katalog_tanaman.katalog_tanaman;
```

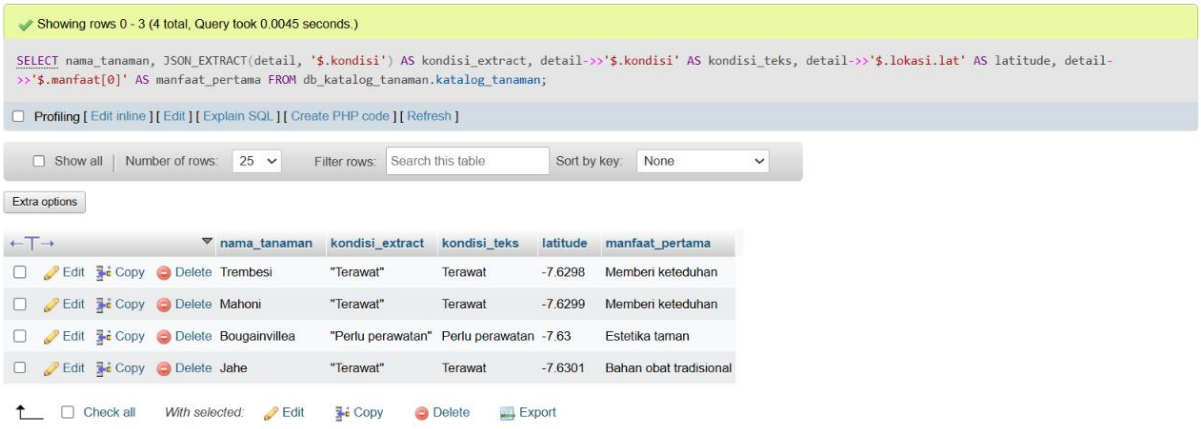


Gambar 4. Isi tabel katalog_tanaman setelah penyimpanan

2.3 Pembacaan Data JSON

Nilai di dalam kolom detail dibaca dengan JSON_EXTRACT() dan operator ->>. JSON_EXTRACT() menghasilkan nilai dalam format JSON sehingga bertanda kutip, sedangkan ->> menghasilkan teks biasa tanpa tanda kutip. Path $.lokasi.lat membaca atribut bersarang, dan $.manfaat[0] mengambil elemen pertama dari array manfaat. Nilai latitude Bougainvillea tampil -7.63 karena angka pada JSON dinormalisasi, nilainya tetap sama.

```
SELECT
  nama_tanaman,
  JSON_EXTRACT(detail, '$.kondisi') AS kondisi_extract,
  detail->>'$.kondisi'              AS kondisi_teks,
  detail->>'$.lokasi.lat'           AS latitude,
  detail->>'$.manfaat[0]'           AS manfaat_pertama
FROM db_katalog_tanaman.katalog_tanaman;
```



Gambar 5. Pembacaan atribut JSON dengan JSON_EXTRACT() dan ->>

2.4 Pencarian Data JSON

Pencarian dilakukan dengan tiga cara berbeda pada isi kolom detail.

a. Pencarian berdasarkan nilai atribut

Mencari tanaman yang atribut kondisi-nya bernilai "Perlu perawatan". Hasilnya satu baris, yaitu Bougainvillea.

```
SELECT id_tanaman, nama_tanaman, detail->>'$.kondisi' AS kondisi
FROM db_katalog_tanaman.katalog_tanaman
WHERE detail->>'$.kondisi' = 'Perlu perawatan';
```

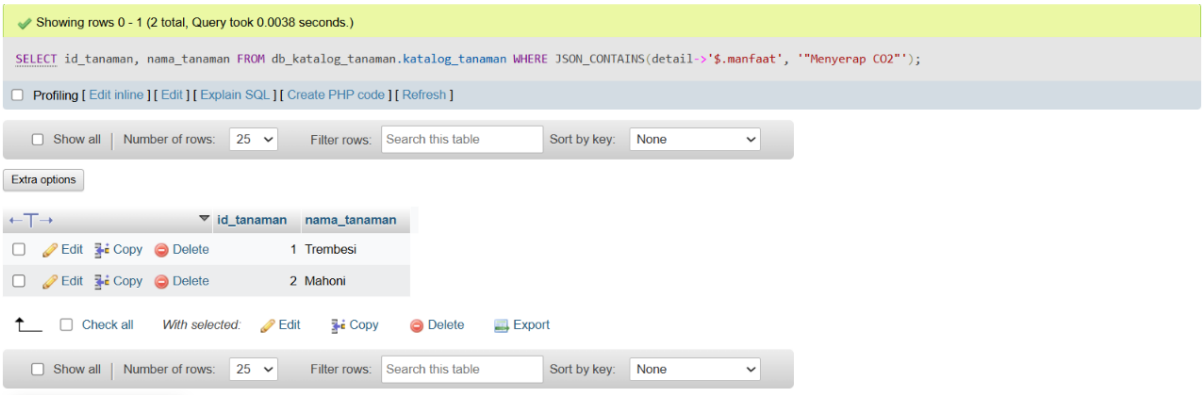


Gambar 6. Pencarian berdasarkan nilai atribut kondisi

b. Pencarian nilai di dalam array

JSON_CONTAINS() digunakan untuk mencari tanaman yang array manfaat-nya memuat "Menyerap CO2". Hasilnya dua baris, yaitu Trembesi dan Mahoni. Nilai yang dicari ditulis dengan tanda kutip ganda di dalam tanda kutip tunggal karena dicocokkan sebagai nilai JSON.

```
SELECT id_tanaman, nama_tanaman
FROM db_katalog_tanaman.katalog_tanaman
WHERE JSON_CONTAINS(detail->'$.manfaat', '"Menyerap CO2"');
```

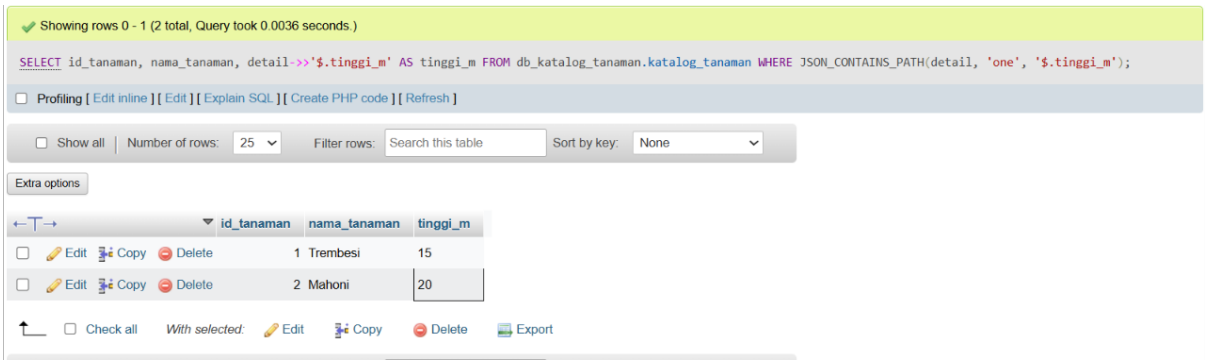


Gambar 7. Pencarian nilai di dalam array dengan JSON_CONTAINS()

c. Pencarian berdasarkan keberadaan atribut

JSON_CONTAINS_PATH() digunakan untuk mencari tanaman yang memiliki atribut tinggi_m. Hasilnya dua baris, yaitu Trembesi (15) dan Mahoni (20). Hanya pohon yang memiliki atribut ini, tanpa ada kolom kosong pada tanaman lain.

```
SELECT id_tanaman, nama_tanaman, detail->>'$.tinggi_m' AS tinggi_m
FROM db_katalog_tanaman.katalog_tanaman
WHERE JSON_CONTAINS_PATH(detail, 'one', '$.tinggi_m');
```



Gambar 8. Pencarian berdasarkan keberadaan atribut dengan JSON_CONTAINS_PATH()

2.5 Perubahan Atribut JSON dengan JSON_SET

JSON_SET() mengubah nilai atribut yang sudah ada dan menambahkan atribut baru jika belum ada. Pada tanaman Bougainvillea (id 3), nilai kondisi diubah dari "Perlu perawatan" menjadi "Terawat", dan atribut baru terakhir_disiram ditambahkan. Perubahan ini hanya berlaku pada db_katalog_tanaman dan tidak mengubah db_green_map_pnm.

```
UPDATE db_katalog_tanaman.katalog_tanaman
SET detail = JSON_SET(detail,
      '$.kondisi', 'Terawat',
      '$.terakhir_disiram', '2026-11-11')
WHERE id_tanaman = 3;
```



Gambar 9. Perubahan atribut JSON dengan JSON_SET() (1 row affected)

Hasil perubahan diperiksa dengan SELECT. Kondisi Bougainvillea menjadi Terawat dan atribut terakhir_disiram bernilai 2026-11-11.

```
SELECT id_tanaman, nama_tanaman,
       detail->>'$.kondisi'          AS kondisi,
       detail->>'$.terakhir_disiram' AS terakhir_disiram
FROM db_katalog_tanaman.katalog_tanaman
WHERE id_tanaman = 3;
```

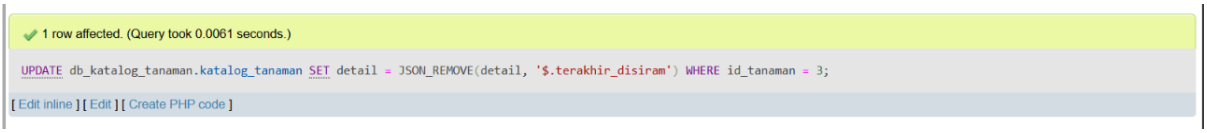


Gambar 10. Hasil perubahan atribut setelah JSON_SET()

2.6 Penghapusan Atribut JSON dengan JSON_REMOVE

JSON_REMOVE() menghapus satu atribut dari dalam JSON tanpa mengubah atribut lainnya. Atribut terakhir_disiram yang sebelumnya ditambahkan dihapus kembali dari Bougainvillea.

```
UPDATE db_katalog_tanaman.katalog_tanaman
SET detail = JSON_REMOVE(detail, '$.terakhir_disiram')
WHERE id_tanaman = 3;
```



Gambar 11. Penghapusan atribut JSON dengan JSON_REMOVE() (1 row affected)

Hasil penghapusan diperiksa dengan SELECT. Atribut terakhir_disiram bernilai NULL dan JSON_CONTAINS_PATH() menghasilkan 0, artinya atribut sudah tidak ada. Atribut kondisi tetap Terawat, sehingga hanya atribut yang dituju yang terhapus.

```
SELECT id_tanaman, nama_tanaman,
       detail->>'$.kondisi'          AS kondisi,
       detail->>'$.terakhir_disiram' AS terakhir_disiram,
       JSON_CONTAINS_PATH(detail,  'one',  '$.terakhir_disiram')  AS
ada_atribut
FROM db_katalog_tanaman.katalog_tanaman
WHERE id_tanaman = 3;
```



Gambar 12. Hasil penghapusan atribut setelah JSON_REMOVE()